

Разработка real-time интерфейса с поддержкой горизонтального масштабирования WebSocket-соединений для системы отслеживания задач

Защита выпускной квалификационной работы

Студент: Зайцев А. С., группа 41313

Руководитель: ст. преподаватель С. А. Рогачев

Санкт-Петербург, 2026

Проблема и цель

Single-node SignalR ограничивает горизонтальное масштабирование

Проблема

Сведения о подключениях и группах SignalR хранятся в памяти процесса.

При нескольких экземплярах backend событие, созданное на одном узле, не доходит до клиента, подключенного к другому узлу.

Real-time пространство фрагментируется на независимые острова.

Цель работы

Real-time интерфейс системы отслеживания задач, в котором WebSocket-соединения обслуживаются несколькими экземплярами app-арі, а события об изменении сущностей доставляются клиентам независимо от узла подключения.

Анализ подходов к масштабированию

Шесть рассмотренных вариантов; обоснован выбор Redis backplane

Один экземпляр backend
Нет масштабирования; один узел — точка отказа

Несколько экземпляров без backplane
Real-time пространство фрагментируется по узлам

Sticky sessions
Удерживают клиента на узле, но не доставляют события на другие

Broker / event bus
Даёт межузловую доставку, но требует схем событий и persistence

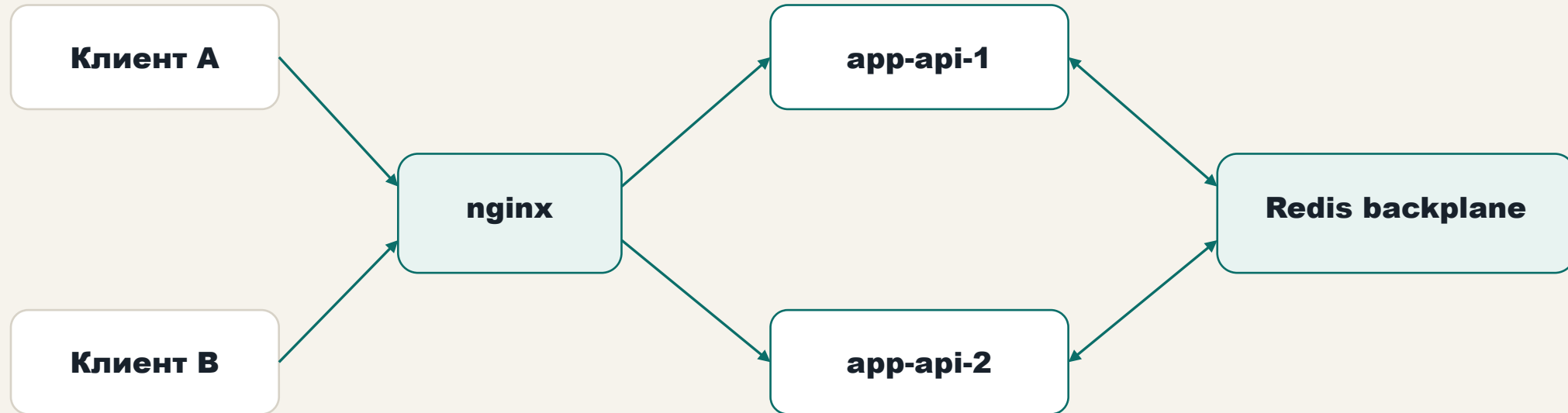
Managed real-time service
Уменьшает контроль над self-hosted контуром

**Redis backplane для SignalR —
выбрано
self-hosted, без перестройки
логики, transient pub/sub**

Критерии выбора: совместимость с self-hosted стендом, минимальная перестройка доменной логики, наличие штатной поддержки в ASP.NET Core SignalR, пригодность для transient UI-событий и воспроизводимость на локальном Docker Compose стенде.

Целевая архитектура

Два экземпляра app-api за nginx, общий Redis backplane для межузловой доставки



Событие публикуется в Redis · доставляется обоим узлам · группы SignalR остаются локальным runtime-состоянием

Оба экземпляра подключены к общей PostgreSQL; на схеме показан только канал real-time-доставки.

Что сделано

Backend-конфигурация, диагностика, стенд и проверочный клиент

1 · Redis backplane для SignalR

AddStackExchangeRedis с prefix app-api-realtime;
включается переменной REDIS_CONNECTION_STRING
— один и тот же код в обоих режимах.

3 · Multi-instance стенд

docker-compose.thesis.yml: app-api-1, app-api-2, Redis,
PostgreSQL, nginx; override .no-backplane.yml для
воспроизведения исходной проблемы.

2 · Диагностический контур

SERVER_INSTANCE_ID,
GetConnectionDiagnosticsAsync(), логирование
подключения, вступления в группу и отправки события.

4 · Проверочный клиент на Node.js

realtime-scaleout-check.mjs: одиночная доставка, серия,
rejoin, failover.

Программа испытаний

Два режима по одному коду · четыре сценария проверки

Два режима

- **Multi-instance без Redis backplane — воспроизведение проблемы**
- **Multi-instance с Redis backplane — проверка решения**

Один и тот же код, один и тот же сценарий; отличие — значение REDIS_CONNECTION_STRING.

Четыре сценария

- Одиночная cross-node доставка
- Серия из 5 итераций (THESIS_ITERATIONS)
- Rejoin после разрыва соединения
- Failover после остановки узла

Главный результат

Один код и один сценарий — разница только в наличии Redis-канала

Без backplane

**ReceiveReportPatch
timed out · 10 000 мс**

**Событие, созданное на app-api-1,
не дошло до клиента на app-api-2**

С backplane

**ok: true · ReceiveReportPatch
получен
deliveryLatencyMs ≈ 9,5 мс**

**Клиент на app-api-2 получил
событие с app-api-1**

Сопоставление двух режимов на одном стенде показывает причинную связь между включением межузлового канала SignalR и успешной доставкой события.

Дополнительные проверки

Повторяемость, повторное вступление в группу и восстановление после отказа узла

Серия 5 итераций

successful 5 / 5

min 5,5 мс

avg 8,4 мс

p50 9,1 мс

p95 11,6 мс

Подтверждена повторяемость доставки.

Rejoin после разрыва

Новый connectionId
повторный
JoinReportGroupAsync

rejoin 12,9 мс

delivery 6,6 мс

Подтверждена повторная подписка.

Failover узла

Контейнер app-api-2 остановлен
клиент переподключился через
nginx к app-api-1

reconnect 240,3 мс

delivery 7,3 мс

Подтверждено восстановление после отказа узла.

Вывод

Ограничение single-node WebSocket-архитектуры устранено за счёт распределённого real-time контура; тезис работы подтверждён экспериментально.

Проблема

Single-node SignalR: события не пересекают границу узла.

Решение

SignalR + Redis backplane + nginx + два app-ар; общий код, конфигурация через переменную окружения.

Доказательство

**Delivery (без / с) · серия 5 / 5
· rejoin · failover —
воспроизводимый Docker
Compose стенд.**